ARITHMETIC EXPRESSION COMPILER

A PROJECT REPORT

submitted by

Group 1

Ayrin Anna Binu
AJC24CS071

Angela Ann Byju
AJC24CS050

Ann Maria Mathew
AJC24CS056

Anns Thomas
AJC24CS057

Alwin Pramod-
AJC24CS042

Albert Joseph
AJC24CS031

to

the APJ Abdul Kalam Technological University
in partial fulfillment of the requirement for the award of the Degree

of

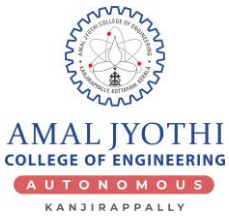
Bachelor of Technology

in

Computer Science and Engineering

Under the guidance of

Ms. TINTU ALPHONSA THOMAS
Assistant Professor



Department of Computer Science and Engineering
Amal Jyothi College of Engineering(Autonomous)
Kanjirappally-686518

October 2026

Project Report: Design and Development of an Arithmetic Expression Compiler

Course: Compiler Design (24CST303)

Executive Summary

Compilers are essential software systems that convert high-level source code into executable instructions or computed results through a series of well-defined stages. This report presents the theoretical foundations, formal grammar specifications, modular architecture, and execution process of an **Arithmetic Expression Compiler** developed using two complementary approaches:

- **C-based CLI implementation:** Focuses on low-level parsing techniques, manual dynamic memory allocation and management, concrete Abstract Syntax Tree (AST) construction, and static semantic validation.
- **Java Swing GUI implementation**: Provides a desktop graphical interface for entering arithmetic expressions, executing the existing C-based compiler, and displaying compilation results, including syntax analysis, Abstract Syntax Tree (AST) output, and semantic-check messages.

1. Problem Statement and Objectives

1.1 Problem Statement

Evaluating arithmetic expressions in programming environments requires validating that the input follows formal syntax rules, including operator precedence, associativity, and balanced delimiters. It is also necessary to perform static semantic checks, such as verifying that variables are defined and detecting division-by-zero errors, before carrying out computations. Using direct execution methods, such as JavaScript's eval () function, introduces significant security vulnerabilities, including the risk of code injection. These methods also provide limited error reporting and bypass the structured syntax analysis required in a compiler. Therefore, a dedicated arithmetic expression compiler is needed to validate expressions, identify errors accurately, and evaluate them through a structured compilation process.

1.2 Objectives

1. Develop a clear, unambiguous, non-left-recursive Context-Free Grammar (CFG) that defines mathematical operators such as *, /, +, and -, while correctly enforcing operator precedence and left associativity.
2. Design a Lexical Analyzer (Scanner) that converts raw character sequences into categorized tokens and identifies lexical errors during scanning.
3. Implement a Recursive-Descent LL(1)/LL(2) Parser that checks syntactic correctness while generating both a Concrete Parse Tree and an Abstract Syntax Tree (AST).
4. Integrate Semantic Analysis to maintain symbol definitions, verify identifier usage, and detect invalid operations, including division by zero.
5. Evaluate valid arithmetic expressions using a recursive AST tree-walking interpreter.
6. Build an intuitive Graphical User Interface (GUI) that helps students, examiners, and software developers understand the internal stages of compilation.

2. Theoretical Compiler Architecture and Phases

A conventional compiler consists of two major components: the Analysis Phase (Front-End) and the Synthesis Phase (Back-End). The implementations examined in this project primarily concentrate on the front-end processing pipeline.

2.1 Breakdown of Compiler Phases

Phase	Input	Output	Main Responsibilities in This Project
Lexical Analysis	Character stream	Token stream (type, lexeme, position)	Removes whitespace, identifies numbers, identifiers and operators, and reports invalid characters.
Syntax Analysis	Token stream	Parse Tree and AST	Applies grammar rules, checks matching parentheses, and constructs a hierarchical tree that reflects operator precedence.
Semantic Analysis	AST	Annotated AST and Symbol Table	Checks whether variables are declared before use, verifies assignment order, and detects division-by-zero errors.
Evaluation / Execution	Annotated AST and Symbol Table	Computed numerical result	Traverses the AST recursively to calculate expression values without exposing the system to the security risks of direct code execution.

3. Formal Grammar Definition and Transformations

3.1 Original Mathematical Grammar (Ambiguous Form)

The initial grammar used to describe arithmetic expressions is ambiguous and contains left-recursive production rules.

Grammar:

```
E -> E + E | E - E | E * E | E / E | ( E ) | id | num
```

Limitations of the Original Grammar:

- **Ambiguity:** An expression such as `2 + 3 * 4` can produce multiple parse trees because the grammar does not explicitly define operator precedence.
- **Left Recursion:** The production `E -> E + ...` directly refers to `E` on its left-hand side within its own expansion. This can cause infinite recursion in top-down recursive-descent parsers.

3.2 Revised Unambiguous Grammar (Right-Recursive Form)

To support deterministic top-down parsing, the grammar is reorganized into separate non-terminals according to operator precedence. This structure removes ambiguity and eliminates direct left recursion.

- **Expression (`Expr / E`):** Handles addition (`+`) and subtraction (`-`), which have lower precedence.
- **Term (`Term / T`):** Represents multiplication (`*`) and division (`/`), which have higher precedence than addition and subtraction.
- **Unary (`Unary`):** Processes unary positive (`+`) and negative (`-`) operators applied to operands.
- **Primary / Factor (`Primary / F`):** Represents basic values, including numeric literals (`NUMBER`), identifiers (`IDENTIFIER`), and parenthesized expressions (`( Expr )`).

Productions:

```
Statement  -> IDENTIFIER = Expr | Expr
Expr       -> Term ExprTail
ExprTail   -> + Term ExprTail | - Term ExprTail | ε
Term       -> Unary TermTail
TermTail   -> * Unary TermTail | / Unary TermTail | ε
Unary      -> + Unary | - Unary | Primary
Primary    -> NUMBER | IDENTIFIER | ( Expr )
```

3.3 Preserving Left Associativity

Right-recursive grammar rules naturally process repeated operators through recursive tails. However, the resulting expression must still follow left associativity for addition, subtraction, multiplication, and division.

For example, the expression `10 - 3 - 2` must be interpreted as:

```
(10 - 3) - 2 = 5
```

It should not be evaluated as `10 - (3 - 2)`, which produces a different result.

- **Implementation in C:** The parser processes successive operators iteratively. When it encounters `TOK_PLUS` or `TOK_MINUS`, it repeatedly updates the accumulated left-hand expression using `ast_create_operator(op, left, right)`. This ensures that each new operation is applied to the result of the previous operation, preserving left associativity.

- In the JavaScript version, the `chain()` function converts the right-recursive accumulator into a left-associative form using a `while` loop, producing the result $(10 - 3) - 2 - 5$.

3.4 FIRST and FOLLOW Sets

Non-Terminal	FIRST Set	FOLLOW Set
Primary	{ NUMBER, IDENTIFIER, (}	{ , }
Unary	{ +, −, NUMBER, IDENTIFIER, (}	{ +, −, *, /, $,) }
Term	{ +, −, NUMBER, IDENTIFIER, (}	{ +, −, *, / }
TermTail	{ +, −, *, /, ε }	{ +, −, *, /, $,) }
Expr	{ +, −, NUMBER, IDENTIFIER, (}	{ $,) }
ExprTail	{ +, −, *, /, ε }	{ $,) }

3.5 LL(1) Parsing Table for Expression Grammar

Non-Terminal	NUMBER	IDENTIFIER	(	)	+	−	*	/	$
Expr	Term ExprTail	Term ExprTail	Term ExprTail	—	Term ExprTail	Term ExprTail	—	—	—
ExprTail	—	—	—	ε	+ Term ExprTail	− Term ExprTail	—	—	ε
Term	Unary TermTail	Unary TermTail	Unary TermTail	—	Unary TermTail	Unary TermTail	—	—	—
TermTail	—	—	—	ε	ε	ε	* Unary TermTail	/ Unary TermTail	ε
Unary	Primary	Primary	Primary	—	+ Unary	− Unary	—	—	—
Primary	NUMBER	IDENTIFIER	(Expr)	—	—	—	—	—	—

Note: Each cell contains at most one production rule; hence, the expression grammar is an LL(1) grammar. For example, both `IDENTIFIER = Expr` and `Expr` begin with `IDENTIFIER`, and the parser distinguishes them using a two-token lookahead (`IDENTIFIER` followed by `=`), which enables statement selection (LL(2)).

4. System Implementation & Modular Breakdown

4.1 System 1: Java Swing GUI Front End

The Java Swing implementation provides a desktop graphical interface for the Arithmetic Expression Mini-Compiler. Users can enter arithmetic expressions, execute the existing C compiler, and view compilation results. The Java application communicates with the C executable through standard input and output streams.

1. Guiswing.java

- Implements the graphical interface using Java Swing.
- Provides a text field for entering arithmetic expressions.
- Includes Compile Expression and Clear buttons.
- Displays syntax-analysis results, AST output, and semantic-check messages in a text area.
- Provides a user-friendly interface for demonstrating the compiler.

2. Compiler Integration

- Uses Java's "ProcessBuilder" to launch "mini_compiler.exe".
- Sends user-entered expressions to the C compiler through standard input.
- Captures compiler output and displays it in the GUI.
- Enables compilation without manually using the command-line terminal.
- Requires the C executable to be available at the configured path.

3. User Interface Components

- "JFrame" – Main application window.
- "JTextField" – Accepts arithmetic expressions.
- "JButton" – Triggers compilation and clears input.
- "JTextArea" – Displays compiler output.
- Layout managers – Organize the interface components.

4. Compilation Workflow

- The user enters an arithmetic expression.
- The Compile Expression button launches the C executable.
- The C compiler performs lexical analysis, syntax analysis, AST construction, and semantic checks.
- The compiler generates output and diagnostic messages.
- The Java GUI displays the results.

5. Technologies Used

- Programming Language: Java
- GUI Framework: Java Swing

- Compiler Implementation: C
- C Compiler: GCC
- Development Environment: Visual Studio Code
- Version Control: Git and GitHub

6. Current Implementation Scope

The Java Swing application acts as a graphical interface for the existing C compiler. Lexical analysis, parsing, AST construction, and semantic checks remain implemented in C. Java launches the compiler and displays its output without independently implementing these stages.

4.2 System 2: C CLI Mini-Compiler

The C implementation is organized into modular header and source files, following the division of work among university team members.

1. **lexer.h & lexer.c (Member 2)**
   - Implements `next_token()` using character classification functions (`isalpha`, `isdigit`, `isspace`).
   - Supports recognition of floating-point numbers with single-decimal validation.
   - Returns a Token structure containing `TokenType`, `lexeme`, and `position`.

2. **parser.h & parser.c (Member 3)**
   - Implements recursive descent parsing functions `E()`, `T()`, and `F()`.
   - Builds the ASTNode during parsing.
   - Frees allocated AST nodes when a syntax error is detected, preventing memory leaks.

3. **ast.h & ast.c (Member 4)**
   - Defines a dynamic ASTNode structure with fields type, value, left, and right.
   - Provides `ast_print()` to display an indented terminal tree.
   - Implements `ast_semantic_check()` to validate identifier names and to detect division by zero (/ 0).
   - Includes `ast_free()` for post-order deallocation of the tree.

4. **main.c**
   - Implements an interactive command-line interface that reads an expression using `fgets()`, parses it, prints the AST, and performs semantic checks.

5. Comparative Evaluation

Feature	CD-compiler (Nourin - Web/JS)	mini-compiler (Nidhin - C)
Platform / Interface	Web-based graphical user interface (HTML/CSS)	Terminal-based command-line interface (CLI)
Language & Tooling	Vanilla ES6+ JavaScript	C (C99 / C11, standard library)
Memory Management	Automatic garbage collection	Explicit dynamic memory management (malloc / free)
Lookahead	LL(1) for expressions, LL(2) for statements	LL(1) single-token lookahead
Multi-Statement Support	Yes (one statement per line, with variable assignments)	No (single expression per execution run)
Symbol Table	Complete symbol table tracking definitions, uses, and values	Minimal (validates identifier token syntax)
AST & Parse Tree	Builds and displays both the parse tree and the AST	Builds AST only
Evaluation	Fully computes the value of expressions recursively	Focuses on compilation front-end analysis
Error Feedback	Provides inline diagnostics with column numbers and helpful hints	Displays terminal error messages with character offset

6 . Test Suite & Verification Results

6.1 Test Scenarios & Outputs

- **Valid Test Scenarios and Outputs**

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: 2+3

Syntax analysis passed.

ABSTRACT SYNTAX TREE (AST)
--------------------------
OPERATOR: +
  NUMBER: 2
  NUMBER: 3

SEMANTIC CHECKS
---------------
Semantic checks passed.

Compilation front-end completed successfully.
```

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: 5-2*4

Syntax analysis passed.

ABSTRACT SYNTAX TREE (AST)
--------------------------
OPERATOR: -
  NUMBER: 5
  OPERATOR: *
    NUMBER: 2
    NUMBER: 4

SEMANTIC CHECKS
---------------
Semantic checks passed.

Compilation front-end completed successfully.
```

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: 3*4

Syntax analysis passed.

ABSTRACT SYNTAX TREE (AST)
--------------------------
OPERATOR: *
  NUMBER: 3
  NUMBER: 4

SEMANTIC CHECKS
---------------
Semantic checks passed.

Compilation front-end completed successfully.
```

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: 8/2

Syntax analysis passed.

ABSTRACT SYNTAX TREE (AST)
--------------------------
OPERATOR: /
  NUMBER: 8
  NUMBER: 2

SEMANTIC CHECKS
---------------
Semantic checks passed.

Compilation front-end completed successfully.
```

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: (2+3)*4

Syntax analysis passed.

ABSTRACT SYNTAX TREE (AST)
--------------------------
OPERATOR: *
  OPERATOR: +
    NUMBER: 2
    NUMBER: 3
  NUMBER: 4

SEMANTIC CHECKS
---------------
Semantic checks passed.

Compilation front-end completed successfully.
```

- **Invalid Test Scenarios and Outputs**

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: 2+
Syntax error at position 2: Expected identifier, number, or '('

Compilation stopped: syntax analysis failed.
```

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: *3
Syntax error at position 0: Expected identifier, number, or '(' (found '*')

Compilation stopped: syntax analysis failed.
```

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: (2+3
Syntax error at position 4: Expected closing ')'

Compilation stopped: syntax analysis failed.
```

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: 2*/3
Syntax error at position 2: Expected identifier, number, or '(' (found '/')

Compilation stopped: syntax analysis failed.
```

```
C:\Users\DELL\Downloads\Arthematic exp\member4_ast_semantic_project>.\mini_compiler.exe
==================================
  ARITHMETIC EXPRESSION MINI COMPILER
==================================
Enter an expression: 2 3
Syntax error at position 2: Unexpected token after expression (found '3')

Compilation stopped: syntax analysis failed.
```

7. Conclusion & Future Enhancements

The two evaluated projects represent a complete academic front-end compiler pipeline:

1. **The C implementation** demonstrates how traditional system compilers handle lexical streams, manual memory management using pointers, and the construction of abstract syntax trees (ASTs) in memory.
2. **The Web GUI implementation** provides an interactive platform for inspecting token streams, tree generation, symbol table evolution, and AST interpretation.

Future Enhancements:

- Extend the grammar to handle comparison operators (<, >, ==) and conditional constructs (if–else).
- Introduce intermediate-code generation using Three-Address Code (TAC) or quadruples.
- Add target-code generation for platforms such as MIPS, x86-64 assembly, or WebAssembly (Wasm).